PTEC103

API Gateway seguro con FastAPI y HashiCorp Vault

Proyecto N°2

Nombre integrantes:

- Sebastián Cuesta Moya (21.918.168-K)
- Nicolás Damián Medina González (22.429.927-3)

NRC: 8481
Profesor: Ernesto Vivanco Tapia

4 de noviembre de 2026

Índice

1. Introducción **2**

1.1. Contexto y motivación 2

1.2. Objetivos 2

1.3. Tecnologías utilizadas 3

1.4. Estructura del proyecto 3

1.5. Arquitectura general 3

1.6. Organización del informe 4

2. Desarrollo **4**

2.1. Explicación del primer código: `gateway.py` 4

2.2. Explicación del segundo código: `auth-service.py` 7

2.3. Explicación del tercer código: `backend_api.py` 9

2.4. Explicación del paso a paso 10

2.5. Justificación: por qué es la forma correcta de realizarlo 13

3. Conclusión **15**

1. Introducción

1.1. Contexto y motivación

Las aplicaciones web modernas rara vez se construyen como un único bloque. Lo habitual es separar las responsabilidades en servicios independientes (autenticación, lógica de negocio, almacenamiento de datos) que se comunican entre sí mediante HTTP. Esta separación mejora la mantenibilidad, pero introduce un problema de seguridad: ¿cómo evitar que cualquier cliente invoque directamente cada servicio? y, además, ¿cómo gestionar de forma segura las credenciales que esos servicios intercambian?

Craft & Beer es una tienda de cervezas artesanales cuyo frontend (HTML y JavaScript) consume una API para mostrar el inventario de productos. Para proteger esa API se implementó el patrón **API Gateway**: un único punto de entrada que valida la identidad del usuario antes de enrutar la petición hacia el backend, apoyándose en un servicio de autenticación propio y en **HashiCorp Vault** para custodiar los secretos del sistema.

1.2. Objetivos

Objetivo general. Implementar y documentar un API Gateway seguro que proteja el acceso a la API de inventario de Craft & Beer.

Objetivos específicos.

- Centralizar el acceso en un único punto de entrada (el gateway), de modo que el backend nunca quede expuesto directamente al cliente.
- Emitir y validar tokens de acceso de corta duración mediante un servicio de autenticación independiente.
- Almacenar los secretos compartidos entre servicios en HashiCorp Vault, evitando escribirlos en el código fuente.
- Garantizar que el servicio de autenticación y el backend solo respondan a solicitudes originadas en el gateway.
- Propagar la identidad del usuario autenticado hacia el backend.
- Documentar el procedimiento de puesta en marcha de los tres servicios.

1.3. Tecnologías utilizadas

Tabla 1: Tecnologías empleadas y su rol dentro del proyecto.

Tecnología	**Rol en el proyecto**
Python 3.10 o superior	Lenguaje de los tres servicios (la sintaxis `str | None` del backend lo requiere).
FastAPI	Framework para construir las APIs REST, con validación y documentación automática.
Uvicorn	Servidor ASGI que ejecuta cada aplicación FastAPI.
httpx	Cliente HTTP asíncrono que usa el gateway para comunicarse con Vault, el Auth Service y el backend.
Pydantic	Validación de los cuerpos de las solicitudes (`LoginRequest`, `IntrospectionRequest`).
HashiCorp Vault	Almacén centralizado de secretos (motor KV versión 2).
PowerShell	Terminal desde la cual se definen las variables de entorno y se lanzan los servicios.

1.4. Estructura del proyecto

El proyecto se organiza en una carpeta raíz llamada `apigateway`, con una subcarpeta por cada servicio:

```
apigateway/
|-- auth-service/
| ‘-- auth-service.py
|-- fastapi/
| ‘-- backend_api.py
‘-- gateway/
    ‘-- gateway.py
```

Cada carpeta contiene un único archivo que corresponde a un servicio independiente. Esto permite ejecutarlos, reiniciarlos y, en un escenario real, desplegarlos por separado.

1.5. Arquitectura general

El sistema está formado por tres servicios propios y un componente externo (Vault). El cliente web solo conoce al gateway y al endpoint de login; el backend queda detrás del gateway y no se comunica nunca directamente con el navegador.

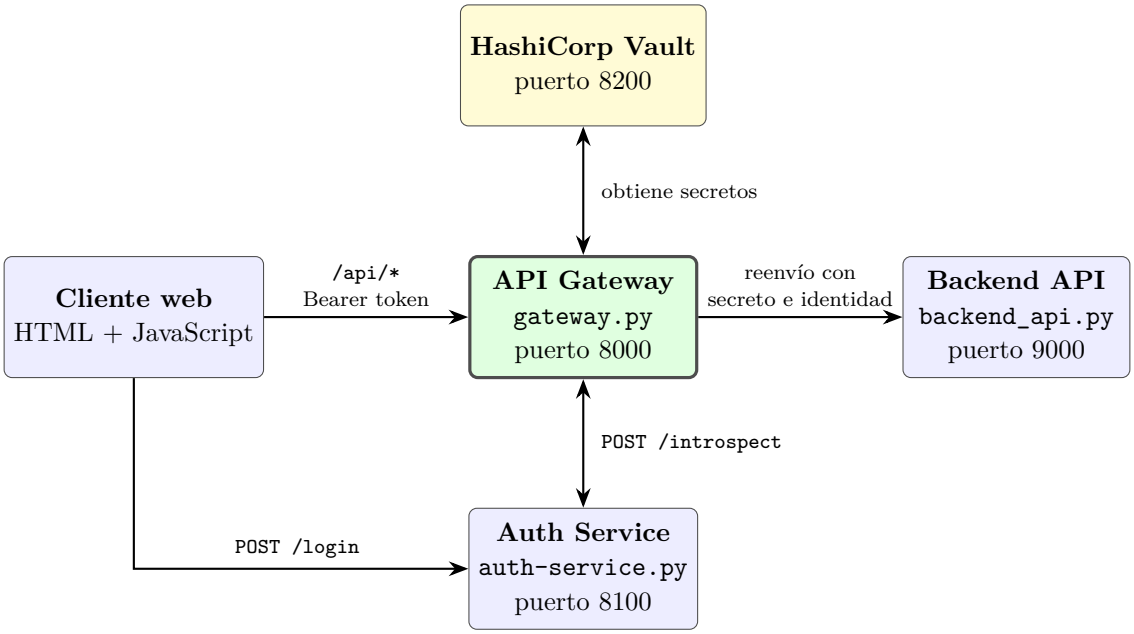


Figura 1: Arquitectura general del sistema Craft & Beer.

Tabla 2: Servicios que componen el sistema.

Servicio	**Archivo**	**Puerto**	**Responsabilidad**
API Gateway	`gateway.py`	8000	Punto de entrada: autentica y enruta hacia el backend.
Auth Service	`auth-service.py`	8100	Login, emisión e introspección de tokens.
Backend API	`backend_api.py`	9000	Lógica de negocio: inventario de cervezas.
HashiCorp Vault	(externo)	8200	Almacén de secretos (puerto por defecto en el gateway).

1.6. Organización del informe

En la sección 2 se explica cada uno de los tres códigos, seguido del paso a paso para ejecutarlos y de la justificación de las decisiones de diseño y de seguridad. La sección 3 presenta las conclusiones, las limitaciones y las mejoras posibles.

2. Desarrollo

2.1. Explicación del primer código: `gateway.py`

El archivo `gateway.py` implementa el API Gateway. Es el único servicio al que accede el frontend y concentra tres funciones: obtener los secretos desde Vault, autenticar al usuario consultando al Auth Service y reenviar la petición al backend con la identidad ya verificada.

2.1.1. Configuración inicial y CORS

Se crea la aplicación FastAPI y se habilita el middleware `CORSMiddleware`. Sin esta configuración, el navegador bloquearía las llamadas que hace el JavaScript de las páginas HTML, ya que el frontend y el gateway se sirven desde orígenes distintos.

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

security = HTTPBearer(auto_error=False)
```

El esquema `HTTPBearer` extrae el token del encabezado `Authorization: Bearer TOKEN`. Con `auto_error=False` FastAPI no responde por sí mismo cuando falta el encabezado, lo que permite devolver un error 401 con un mensaje propio.

2.1.2. Variables de entorno y token de Vault

Las direcciones de los demás servicios se leen desde variables de entorno, con valores por defecto pensados para ejecutar todo en `localhost`. En cambio, `VAULT_TOKEN` no tiene valor por defecto: si no está definido, el servicio se detiene al arrancar.

```
AUTH_SERVICE_URL = os.getenv("AUTH_SERVICE_URL", "http://127.0.0.1:8100")
VAULT_ADDR = os.getenv("VAULT_ADDR", "http://127.0.0.1:8200")
BACKEND_URL = os.getenv("BACKEND_URL", "http://127.0.0.1:9000")

VAULT_TOKEN = os.getenv("VAULT_TOKEN")

if not VAULT_TOKEN:
    raise RuntimeError("CRÍTICO: VAULT_TOKEN no está configurado.")
```

2.1.3. Obtención de secretos desde Vault

La función `get_gateway_secrets` consulta la API HTTP de Vault enviando el token en el encabezado `X-Vault-Token`. La ruta `secret/data/gateway` corresponde al motor KV versión 2, que anida los valores bajo la clave `data`, por lo que el resultado se lee en `["data"]["data"]`.

```
async def get_gateway_secrets():
    url = f"{VAULT_ADDR}/v1/secret/data/gateway"
    headers = {"X-Vault-Token": VAULT_TOKEN}

    async with httpx.AsyncClient(timeout=5.0) as client:
        response = await client.get(url, headers=headers)

    if response.status_code != 200:
        raise HTTPException(status_code=500, detail="No fue posible conectarse o autenticarse con
            ↪ Vault")
```

```
    vault_response = response.json()
    return vault_response["data"]["data"]
```

El secreto almacenado en `secret/gateway` contiene dos claves que usa el gateway: `auth_introspection_secret` (para hablar con el Auth Service) y `backend_shared_secret` (para hablar con el backend).

2.1.4. Autenticación del cliente

La función `authenticate_client` se declara como dependencia (`Depends`) de la ruta de proxy, de modo que se ejecuta antes de reenviar cualquier solicitud. Sus pasos son:

1. Verifica que exista un token Bearer; si no, responde 401.
2. Obtiene los secretos desde Vault.
3. Consulta al Auth Service en `/introspect`, enviando el token del usuario y el secreto de introspección en el encabezado `X-Gateway-Auth-Secret`.
4. Si el Auth Service no responde, devuelve 502; si el token no está activo, devuelve 401.
5. Si todo es correcto, retorna la identidad del usuario junto con el secreto que se usará frente al backend.

```
async def authenticate_client(credentials: HTTPAuthorizationCredentials = Depends(security)):
    if not credentials:
        raise HTTPException(status_code=401, detail="Bearer token requerido")

    gateway_secrets = await get_gateway_secrets()
    introspection_secret = gateway_secrets["auth_introspection_secret"]

    try:
        async with httpx.AsyncClient(timeout=5.0) as client:
            response = await client.post(
                f"{AUTH_SERVICE_URL}/introspect",
                json={"token": credentials.credentials},
                headers={"X-Gateway-Auth-Secret": introspection_secret}
            )
    except httpx.RequestError:
        raise HTTPException(status_code=502, detail="Error consultando al Auth Service")

    identity = response.json()

    if not identity.get("active", False):
        raise HTTPException(status_code=401, detail="Token inválido o expirado")

    return {
        "client_id": "frontend-web",
        "user_id": identity["user_id"],
        "username": identity["username"],
        "roles": identity["roles"],
        "backend_secret": gateway_secrets["backend_shared_secret"]
    }
```

2.1.5. Proxy hacia el backend

La ruta `/api/\{path:path\}` captura cualquier subruta y acepta los métodos GET, POST, PUT, PATCH y DELETE. Por ejemplo, una llamada a `/api/productos` se traduce en una petición a `http://127.0.0.1:9000/productos`. El gateway reenvía el cuerpo y los parámetros de consulta originales, y construye *desde cero* los encabezados que recibirá el backend.

```
@app.api_route("/api/{path:path}", methods=["GET", "POST", "PUT", "PATCH", "DELETE"])
async def proxy(path: str, request: Request, auth=Depends(authenticate_client)):
    target_url = f"{BACKEND_URL}/{path}"
    body = await request.body()

    gateway_headers = {
        "X-Gateway-Secret": auth["backend_secret"],
        "X-Authenticated-Client": auth["client_id"],
        "X-Authenticated-User": auth["username"],
        "X-Authenticated-Roles": ",".join(auth["roles"])
    }

    if "content-type" in request.headers:
        gateway_headers["content-type"] = request.headers["content-type"]
```

Luego realiza la petición con `httpx` (devolviendo 502 si el backend no está disponible) y entrega al cliente el contenido, el código de estado y el tipo de contenido de la respuesta original.

Detalle importante

El token del usuario **no se reenvía** al backend, y los encabezados `X-Authenticated-*` se generan en el gateway a partir de la respuesta del Auth Service. Así, un cliente malicioso no puede falsificar su identidad enviando esos encabezados por su cuenta.

2.2. Explicación del segundo código: `auth-service.py`

El archivo `auth-service.py` implementa el servicio de autenticación. Tiene dos responsabilidades: permitir que un usuario inicie sesión y recibir un token (`/login`) y confirmar al gateway si un token sigue siendo válido (`/introspect`). Al igual que el gateway, habilita CORS para que el formulario de login del frontend pueda llamarlo.

2.2.1. Base de datos simulada y sesiones

Para efectos del proyecto, los usuarios se almacenan en un diccionario en memoria y las sesiones activas en otro. Cada token vive 15 minutos.

```
USERS = {
    "nicolas": {
        "password": "password123",
        "user_id": "C-001",
        "roles": ["cliente", "admin"]
    }
}

SESSIONS = {}
TOKEN_LIFETIME_MINUTES = 15
```

2.2.2. Secreto de introspección

El servicio lee la variable de entorno `AUTH_INTROSPECTION_SECRET`. Su valor debe coincidir con la clave `auth_introspection_secret` guardada en Vault, ya que es la contraseña con la que el gateway demuestra ser quien dice ser. Si la variable no está definida, el servicio no inicia.

```
AUTH_INTROSPECTION_SECRET = os.getenv("AUTH_INTROSPECTION_SECRET")

if not AUTH_INTROSPECTION_SECRET:
    raise RuntimeError("CRÍTICO: AUTH_INTROSPECTION_SECRET no está configurado.")
```

2.2.3. Endpoint de login

Recibe usuario y contraseña (validados con un modelo Pydantic). Si las credenciales son correctas, genera un token aleatorio con `secrets.token_urlsafe(32)`, que equivale a 32 bytes de un generador criptográficamente seguro (256 bits de entropía). Luego registra la sesión con la identidad del usuario y su fecha de expiración, y entrega el token junto con su duración en segundos.

```
@app.post("/login")
def login(request: LoginRequest):
    user = USERS.get(request.username)
    if not user or user["password"] != request.password:
        raise HTTPException(status_code=401, detail="Credenciales incorrectas")

    access_token = secrets.token_urlsafe(32)
    expiration = datetime.now(timezone.utc) + timedelta(minutes=TOKEN_LIFETIME_MINUTES)

    SESSIONS[access_token] = {
        "user_id": user["user_id"],
        "username": request.username,
        "roles": user["roles"],
        "expires_at": expiration
    }

    return {
        "access_token": access_token,
        "token_type": "bearer",
        "expires_in": TOKEN_LIFETIME_MINUTES * 60
    }
```

Observe que se usa el mismo mensaje de error tanto si el usuario no existe como si la contraseña es incorrecta, de modo que no se revela qué usuarios están registrados.

2.2.4. Endpoint de introspección

Este endpoint responde a la pregunta «¿este token sigue siendo válido y a quién pertenece?». Solo atiende al gateway: primero compara el encabezado `X-Gateway-Auth-Secret` con el secreto

configurado y, si no coincide, responde 403. Después busca la sesión; si no existe o ya expiró, la elimina y responde `active: false`. Si es válida, devuelve la identidad completa.

```
@app.post("/introspect")
def introspect(request: IntrospectionRequest, x_gateway_auth_secret: str = Header(default="")):
    if not secrets.compare_digest(x_gateway_auth_secret, AUTH_INTROSPECTION_SECRET):
        raise HTTPException(status_code=403, detail="Gateway no autorizado")

    session = SESSIONS.get(request.token)
    if not session or datetime.now(timezone.utc) > session["expires_at"]:
        SESSIONS.pop(request.token, None)
        return {"active": False}

    return {
        "active": True,
        "user_id": session["user_id"],
        "username": session["username"],
        "roles": session["roles"],
        "expires_at": session["expires_at"].isoformat()
    }
```

Detalle importante

FastAPI convierte el parámetro `x_gateway_auth_secret` en el encabezado HTTP `X-Gateway-Auth-Secret`, que es exactamente el que envía el gateway.

2.3. Explicación del tercer código: `backend_api.py`

El archivo `backend_api.py` contiene la API de negocio: entrega el inventario de cervezas que consume el frontend. Su característica principal es que **no confía en nadie salvo en el gateway**.

2.3.1. Secreto compartido con el gateway

Igual que los otros servicios, exige una variable de entorno al iniciar: `INTERNAL_GATEWAY_SECRET`, cuyo valor debe coincidir con la clave `backend_shared_secret` de Vault.

2.3.2. Verificación del origen de la solicitud

La función `verificar_gateway` lee el encabezado `X-Gateway-Secret` y lo compara con el secreto interno. Si no coincide (o no viene), responde 403.

```
INTERNAL_GATEWAY_SECRET = os.getenv("INTERNAL_GATEWAY_SECRET")

if not INTERNAL_GATEWAY_SECRET:
    raise RuntimeError("CRÍTICO: INTERNAL_GATEWAY_SECRET no está configurado en el entorno.")

def verificar_gateway(x_gateway_secret: str = Header(default="")):
    valid = secrets.compare_digest(x_gateway_secret, INTERNAL_GATEWAY_SECRET)
    if not valid:
        raise HTTPException(
            status_code=403,
            detail="Solicitud no autorizada desde Gateway"
```

```
    )
```

2.3.3. Endpoints disponibles

Ambos endpoints incluyen la verificación mediante `dependencies=[Depends(verificar_gateway)]`, por lo que ninguno es accesible sin el secreto del gateway.

- `GET /health`: indica que el servicio está operativo y refleja el cliente autenticado recibido en el encabezado `X-Authenticated-Client`.
- `GET /productos`: retorna el inventario de cervezas junto con la identidad (cliente, usuario y roles) que el gateway propagó.

```
@app.get("/productos", dependencies=[Depends(verificar_gateway)])
def productos(
    x_authenticated_client: str | None = Header(default=None),
    x_authenticated_user: str | None = Header(default=None),
    x_authenticated_roles: str | None = Header(default=None)
):
    return {
        "identidad": {
            "cliente_id": x_authenticated_client,
            "usuario": x_authenticated_user,
            "roles": x_authenticated_roles
        },
        "data": [ ... ]
    }
```

Cada producto tiene los campos `id`, `nombre`, `precio`, `imagen`, `clase` (clase CSS que usa el frontend para la tarjeta) y `stock`. El inventario simulado se resume en la Tabla 3.

Tabla 3: Inventario simulado devuelto por `/productos`.

ID	Producto	Precio	Stock
1	«Hops & Glory» American IPA	3500	10
2	«Nebulosa» Hazy IPA	4200	15
3	«Doble Impacto» Double IPA	4500	8
4	«Despertar» Coffee Porter	3500	12
5	«Abismo» Imperial Stout	4800	5
6	«Oasis» Blonde Ale	2800	20

2.4. Explicación del paso a paso

Los tres servicios se ejecutan al mismo tiempo, cada uno en su propia terminal de PowerShell.

Prerrequisito

HashiCorp Vault debe estar en ejecución (en `http://127.0.0.1:8200`) y contener en `secret/gateway` las claves `auth_introspection_secret` y `backend_shared_secret`. Sus valores deben ser idénticos a los que se entregan por variable de entorno al Auth Service y al backend, respectivamente (Tabla 4).

Tabla 4: Correspondencia entre los secretos de Vault y las variables de entorno.

Clave en Vault	Variable de entorno	Servicio	Uso
`auth_introspection_secret`	`AUTH_INTROSPECTION_SECRET`	Auth Service	Autoriza al gateway a usar `/introspect`.
`backend_shared_secret`	`INTERNAL_GATEWAY_SECRET`	Backend API	Autoriza al gateway a llamar al backend.

2.4.1. Primer paso: levantar el API Gateway

```
cd apigateway\gateway
$env:VAULT_TOKEN="hvs.XXXXXXX"; uvicorn gateway:app --port 8000 --reload
```

El primer comando ubica la terminal en la carpeta del servicio. El segundo define `VAULT_TOKEN` (necesario para que el gateway pueda leer los secretos de Vault) y, tras el punto y coma, inicia Uvicorn. El gateway queda escuchando en el puerto 8000.

2.4.2. Segundo paso: levantar el Auth Service

```
cd apigateway\auth-service
$env:AUTH_INTROSPECTION_SECRET="secreto-seguro-auth-2026"; uvicorn auth-service:app --port 8100
    ↪ --reload
```

Se define el secreto de introspección y se inicia el servicio de autenticación en el puerto 8100. Aunque el guion no es válido en una instrucción `import` normal de Python, Uvicorn carga el módulo a partir de su nombre en texto (`auth-service:app`), por lo que funciona con este nombre de archivo.

2.4.3. Tercer paso: levantar el Backend API

```
cd apigateway\fastapi
$env:INTERNAL_GATEWAY_SECRET="llave-maestra-backend-2026"; uvicorn backend_api:app --port 9000
    ↪ --reload
```

Se define el secreto interno y se inicia el backend en el puerto 9000. A partir de aquí, el sistema completo está operativo.

2.4.4. Anatomía de los comandos

Tabla 5: Componentes de los comandos utilizados.

Fragmento	Función
`cd apigateway\gateway`	Entra en la carpeta del servicio para que Uvicorn encuentre el módulo.
`$env:NOMBRE="valor"`	Define una variable de entorno visible solo para esa sesión de PowerShell.
`;`	Separador de PowerShell: ejecuta el siguiente comando a continuación.
`uvicorn gateway:app`	Ejecuta el objeto `app` del módulo `gateway` (archivo `gateway.py`).
`--port 8000`	Puerto en el que escucha el servicio.
`--reload`	Reinicia el servicio automáticamente al detectar cambios en el código (modo desarrollo).

Sobre el orden de arranque

El orden gateway, Auth Service y backend sigue el recorrido de una petición, de la entrada hacia el interior. Sin embargo, no es técnicamente obligatorio: ningún servicio contacta a otro durante el arranque, solo al recibir peticiones. Lo único necesario es que los tres (y Vault) estén activos antes de la primera solicitud.

2.4.5. Recorrido de una petición

Una vez levantados los servicios, una consulta del inventario sigue estos pasos:

1. El usuario envía sus credenciales a `POST /login` del Auth Service (puerto 8100).
2. El Auth Service valida los datos, crea la sesión y devuelve un token de acceso con vigencia de 900 segundos.
3. El frontend llama a `GET /api/productos` en el gateway (puerto 8000) con el encabezado `Authorization: Bearer TOKEN`.
4. El gateway extrae el token y consulta a Vault los secretos que necesita.
5. El gateway envía `POST /introspect` al Auth Service con el secreto de introspección en `X-Gateway-Auth-Secret`.
6. El Auth Service comprueba ese secreto, busca la sesión y responde con `active: true` y los datos del usuario.
7. El gateway arma los encabezados de confianza (`X-Gateway-Secret` y `X-Authenticated-*`).
8. El gateway reenvía la petición a `http://127.0.0.1:9000/productos`.
9. El backend verifica `X-Gateway-Secret` y responde con el inventario y la identidad recibida.
10. El gateway devuelve al cliente el contenido, el código de estado y el tipo de contenido de esa respuesta.

2.4.6. Ejemplo de verificación

Con los tres servicios activos, el funcionamiento puede comprobarse desde PowerShell:

```
$r = Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8100/login -ContentType "application/
    ↪ json" -Body '{"username":"nicolas","password":"password123"}'

Invoke-RestMethod -Uri http://127.0.0.1:8000/api/productos -Headers @{ Authorization = "Bearer $
    ↪ ($r.access_token)" }
```

La primera línea obtiene un token; la segunda consulta el inventario a través del gateway. Además, FastAPI genera documentación interactiva en la ruta `/docs` de cada servicio.

2.4.7. Respuestas de error esperables

Tabla 6: Códigos de error y su causa.

Código	Origen	Causa
401	Gateway	Falta el token Bearer, o el token es inválido o ya expiró.
403	Backend	El encabezado `X-Gateway-Secret` no coincide: la solicitud no viene del gateway.
500	Gateway	Vault responde con error (por ejemplo, un `VAULT_TOKEN` inválido).
502	Gateway	El Auth Service o el backend no están disponibles.

Para depurar

El gateway no revisa el código de estado de la respuesta del Auth Service, solo el campo `active`. Por eso, si el secreto de introspección no coincide con el de Vault, el Auth Service responde 403, pero el usuario ve un 401 en el gateway aunque su token sea válido. Ante un 401 inesperado conviene revisar primero que ambos secretos sean idénticos.

2.5. Justificación: por qué es la forma correcta de realizarlo

Esta sección explica las razones técnicas detrás del procedimiento seguido en las terminales y del diseño del código.

2.5.1. Tres servicios, tres carpetas, tres terminales

Cada componente tiene una responsabilidad única: el gateway enruta, el Auth Service gestiona identidades y el backend contiene la lógica de negocio. Ejecutarlos como procesos independientes, en puertos distintos, refleja una arquitectura de microservicios: cada pieza puede modificarse, reiniciarse o escalar sin afectar a las demás, y un fallo en una de ellas queda aislado y es fácil de identificar (por ejemplo, un 502 apunta a un servicio caído).

2.5.2. Secretos en variables de entorno y no en el código

Los pasos 1, 2 y 3 entregan los secretos con `\$env:`[1] en lugar de escribirlos dentro de los archivos `.py`. Esto sigue el principio de separar la configuración del código (metodología *Twelve-Factor App*): el código puede compartirse o subirse a un repositorio sin exponer credenciales, y la variable existe solo durante esa sesión de terminal, sin quedar guardada en disco.

Además, cada servicio aplica el principio de *fallar rápido* (*fail-fast*): si falta la variable, lanza un `RuntimeError` y no arranca. Es preferible que el servicio se detenga a que funcione sin protección alguna.

2.5.3. Vault como fuente central de secretos

El gateway no almacena las contraseñas con las que se comunica con el resto del sistema: las pide a Vault en cada petición. Esto aporta varias ventajas: los secretos están en un único lugar controlado, Vault puede auditar quién los consulta, se pueden rotar sin modificar el código y el acceso se restringe mediante el token de Vault. La única credencial que el gateway necesita recibir por fuera es precisamente `VAULT_TOKEN`, la «llave de entrada» a los demás secretos.

Alcance

En este proyecto el Auth Service y el backend reciben su secreto por variable de entorno, no directamente desde Vault; por eso los valores deben mantenerse idénticos en ambos lugares. En un entorno productivo, estos servicios también consultarían a Vault, y el propio token de Vault se obtendría con un mecanismo de identidad de máquina (por ejemplo, AppRole) y no manualmente.

2.5.4. El gateway como único punto de entrada

Centralizar el acceso en el gateway evita repetir la lógica de autenticación en cada endpoint del backend y reduce la superficie de ataque: el cliente solo conoce una dirección, y el backend nunca recibe tráfico directo de Internet. Cualquier cambio futuro (registro de accesos, límites de uso, nuevas políticas) se implementa en un solo lugar.

2.5.5. Tokens opacos con introspección

El Auth Service emite tokens aleatorios sin información interna y guarda la sesión en el servidor; el gateway los valida preguntando al Auth Service (introspección, patrón descrito en el estándar RFC 7662). Frente a un token autofirmado como JWT, esta estrategia permite invalidar una sesión de inmediato, ya que basta con eliminarla del servidor. La vigencia corta (15 minutos) limita el daño si un token se filtra, y `secrets.token_urlsafe` garantiza que no pueda adivinarse.

2.5.6. Defensa en profundidad entre capas

Cada capa verifica que su interlocutor sea el esperado, en lugar de confiar en estar "detrás" de una red segura (principio de *zero trust*). Aunque alguien conociera la dirección del backend o del Auth Service, recibiría un 403 sin el secreto correspondiente.

[1] En PowerShell, `$env:NOMBRE` accede a las variables de entorno del proceso.

Tabla 7: Capas de seguridad del sistema.

Capa	Mecanismo	Amenaza que mitiga
Cliente → Gateway	Token Bearer de 15 minutos	Acceso sin autenticación.
Gateway → Auth Service	Secreto en `X-Gateway-Auth-Secret`	Que un tercero consulte la validez de tokens.
Gateway → Backend	Secreto en `X-Gateway-Secret`	Acceso directo al backend saltándose el gateway.
Gateway → Vault	`VAULT_TOKEN`	Lectura no autorizada de los secretos.
Identidad hacia el backend	Encabezados generados por el gateway	Suplantación de usuario o de roles.

Todas las comparaciones de secretos usan `secrets.compare_digest`, que tarda el mismo tiempo sin importar en qué carácter difieren las cadenas. Con una comparación normal (`==`) un atacante podría medir las diferencias de tiempo para deducir el secreto carácter a carácter (*timing attack*).

2.5.7. Parámetros de Uvicorn

El parámetro `-{}-port` asigna un puerto distinto a cada servicio para que puedan convivir en la misma máquina sin conflictos y coincidan con las direcciones por defecto definidas en el gateway (8100, 8200 y 9000). El parámetro `-{}-reload` reinicia el servicio al guardar cambios, lo que agiliza el desarrollo; en producción se omite. Cabe notar que, como las sesiones del Auth Service viven en memoria, cada reinicio invalida los tokens emitidos hasta ese momento.

3. Conclusión

Se implementó un API Gateway funcional compuesto por tres servicios FastAPI y apoyado en HashiCorp Vault. El resultado cumple con los objetivos planteados: el backend solo es accesible a través del gateway; los usuarios se autentican con tokens de corta duración que se validan mediante introspección; los secretos se gestionan fuera del código fuente; y la identidad del usuario llega al backend por encabezados que el cliente no puede falsificar.

El procedimiento de puesta en marcha, con una terminal por servicio y secretos entregados por variables de entorno, resulta simple de reproducir y refleja buenas prácticas de seguridad: separación de responsabilidades, configuración externa al código, comparación de secretos en tiempo constante y verificación mutua entre capas.

Limitaciones

Al tratarse de un proyecto con fines de demostración, existen aspectos que deberían reforzarse antes de un uso real:

- Los usuarios y contraseñas están en un diccionario en memoria y las contraseñas no están cifradas ni con *hash*.

- Las sesiones se almacenan en memoria: se pierden al reiniciar el servicio y no pueden compartirse entre varias instancias.
- CORS permite cualquier origen (`*`) y la comunicación entre servicios usa HTTP sin cifrado (TLS).
- El gateway propaga los roles al backend, pero no aplica autorización por rol: solo verifica que el usuario esté autenticado.
- Se consulta a Vault en cada petición, lo que añade latencia, y no se captura un fallo de red al conectar con Vault (en ese caso el gateway devuelve un error 500 genérico).

Mejoras futuras

- Guardar las contraseñas con un algoritmo de *hash* robusto (bcrypt o Argon2) en una base de datos real.
- Mover las sesiones a un almacén compartido, como Redis.
- Habilitar TLS entre todos los componentes y restringir los orígenes permitidos en CORS.
- Implementar control de acceso por roles en el gateway.
- Usar AppRole u otro método de autenticación de Vault y mantener los secretos en caché con un tiempo de vida limitado.
- Agregar registro de accesos y límites de uso (*rate limiting*) en el gateway.

En síntesis, el proyecto muestra cómo un API Gateway, junto con un gestor de secretos y un servicio de autenticación dedicado, permite proteger una API de manera ordenada y escalable, sentando una base sólida sobre la cual incorporar las mejoras mencionadas.